Master's Thesis: Working Notes

Daniel Tesfai

September 4, 2026

Contents

---

1 Matching Fixes, Newer Models, and the Effect of Model Size

1.1 Goal

This section addresses three related questions that build on one another. First, whether fixing the paper's matching step alone recovers some of its reported hallucination cases. Second, whether a newer language model produces meaningfully better output on the paper's zero-shot topic labeling task than the original Llama 3 8B baseline. Third, once a newer model family is established as stronger, whether size itself matters within that family, or whether the improvement from Section 1 is mostly a generation effect rather than a scale effect. The first two questions were investigated together and are reported first; the third question follows directly from their result and is reported as its natural continuation.

1.2 Matching Fixes

The paper's matching logic only lowercases the model's answer and strips whitespace, then requires an exact string match against the 19 topic vocabulary. Two of the paper's own documented hallucination cases would survive that check unchanged:

- The model answering `"'software engineering'"` (with literal quote characters)
- The model answering `computer vision` instead of the actual vocabulary term `computer imaging and vision`

1.2.1 Design decision: why not plain fuzzy string matching

A first attempt used raw character-similarity matching (Python's `difflib`). Checked against the real 19-topic vocabulary before building anything further, it fails on the exact case it is meant to fix:

```
"computer vision" vs "computer aided design"       -> similarity 0.722
"computer vision" vs "computer imaging and vision" -> similarity 0.714
```

A naive top-1 similarity match would pick the wrong topic here, since the many `computer ___` topics in this vocabulary collide under plain character similarity.

A second problem is that `computer science` is the paper's single most common hallucination, and it is the ontology's parent category, not one of the 19 valid topics. It still scores 0.68 to 0.73 in similarity to several real topics (`theoretical computer science`, `computer security`, `computer systems`). Auto-resolving it via fuzzy match would fabricate an answer the model never actually gave.

1.2.2 What was built

The matching module (`src/matching.py`) applies the following steps:

1. Canonicalize: lowercase, strip whitespace, strip quote characters and trailing punctuation.
2. Exact match against the 19 topics, same as the paper, with better cleanup applied first.
3. If no exact match is found, apply a word-containment fuzzy match instead of character similarity. A match is only accepted if every word in the candidate appears in exactly one target's word set. For example, the words {computer, vision} are a subset of `computer imaging and vision`'s words {computer, imaging, and, vision}, but not a subset of `computer aided design`'s words {computer, aided, design}.

4. `computer science` is explicitly excluded from step 3, so it always remains classified as a hallucination.

Table 1 shows the fix verified against the paper's own documented examples.

Table 1: Matching fix verified against the paper's documented hallucination examples

Input	Old (exact match only)	New (`matching.py`)
`"'software engineering'"`	hallucination	software engineering
`computer vision`	hallucination	computer imaging and vision
`computer science`	hallucination	hallucination (correctly unresolved)
`quantum computing` (genuine out-of-vocab)	hallucination	hallucination (correctly unresolved)

1.3 Newer Model Quality Check

1.3.1 Setup

- Cloned the paper's official reference implementation for the exact prompt wording, the matching code being improved on, and the reproducible list of the 2500 D3 paper IDs used in the paper.
- Downloaded and filtered the full D3 papers dataset down to those exact 2500 documents, so ground truth CSO subjects are real rather than a fresh random sample.
- Built a script reproducing the paper's exact 3-turn prompt as a genuine multi-turn conversation (each prompt receives a real reply, and replies stay in the context for the next prompt, matching how the paper's original chat session worked), run against the Anthropic API using the model `claude-haiku-4-5-20251001` (Claude Haiku 4.5). All later references to "Claude" in this section refer to this exact model. Every answer is scored both the old way (exact match only) and the new way (`matching.py`), so both fixes can be evaluated on the same run.

1.4 Local Inference Setup for Qwen3-8B

A second, fully local newer model was wanted for the same quality check, at no API cost. Three approaches were attempted before working inference was reached:

1. **llama-cpp-python**: the first choice, since it reuses the GGUF format and has grammar-constrained decoding support for future work. It failed to build because the local machine's Xcode Command Line Tools were missing C++ standard headers entirely, a broken system toolchain not fixable without a full Command Line Tools reinstall.
2. **Ollama**: a prebuilt binary requiring no compilation. This worked, but inspection showed it running entirely on CPU despite the machine having a capable GPU, since the Homebrew-distributed Ollama build does not have Metal support compiled in.

3. **mlx-lm**: Apple's own inference framework, installable via plain `pip` with no compilation step and Metal-accelerated. It required a different model format than GGUF, so the `mlx-community/Qwen3-8B-4bit` checkpoint was downloaded directly.

mlx-lm on the local Mac is what the small-sample testing below (bug-finding, prompt-structure iteration) ran on. The full 2500-document production run afterward moved to a separate, more capable Linux machine, using Ollama as the local backend there (the Mac's earlier CPU-only Ollama problem did not apply on that machine). That run is what Section 1.5 actually reports.

1.4.1 First version: 3-turn prompt

A bug was found on the first real run: Qwen3's default thinking mode produced a reasoning block that was cut off by the token limit before it closed. The original cleanup logic only stripped properly closed reasoning blocks, so the entire unclosed reasoning text became the model's recorded answer, was split on commas, and one fragment coincidentally fuzzy-matched a real topic, producing a false success unrelated to the model's actual intent. This was fixed by disabling thinking mode explicitly in the chat template call, and by making the cleanup function return an empty string, rather than the raw text, whenever an unclosed reasoning block is detected.

After the fix, this version reached 40% success, 60% misclassified, and 0% hallucination on a small sample. The old and new matching gave identical results, since the output was already clean. A notable pattern emerged: the model answered `information technology` 8 out of 20 times, and was wrong every single time, resembling a generic fallback default in the same spirit as the paper's own `computer science` fallback, just landing on a syntactically valid vocabulary term instead of an invalid one.

1.4.2 Final version: single-prompt

The 3-turn structure required 3 model calls per document for only one scored answer, which is wasteful at the full 2500-document scale. The three turns were collapsed into a single combined prompt, reducing the cost to one model call per document. This version reached 55% success, 40% misclassified, and 5% hallucination on the same small sample, with the first genuine hallucination observed (an answer of `machine learning`, which is not one of the 19 topics). The generic-fallback pattern persisted but shifted label, from `information technology` to `information retrieval`.

1.5 Full-Scale Results: Matching Fix and Model Generation Effects

1.5.1 Qwen3-8B, titles, 2500 documents

Table 2 shows the result of running the single-prompt version on the full 2500-document dataset using titles as the document representation, on the Linux machine via Ollama, as noted in Section 1.4.

Table 2: Qwen3-8B (Ollama backend), titles, 2500 documents

	Success	Misclassified	Hallucination
Old matching	63.3%	29.5%	7.2%
New matching	66.6%	30.3%	3.1%

This is better than the paper's Llama 3 8B baseline (50% / 25% / 25%) across every category: higher success, similar misclassification, and less than a third of the hallucination rate.

The matching fix mattered substantially at this scale, not only marginally as the small-sample runs suggested. 103 of the 2500 answers (4.1%) changed category between old and new matching, and all 103 share the identical raw answer `computer vision`, indicating a systematic model habit rather than a one-off error. Under old matching all 103 were hallucinations. Under new matching, 83 became successes and 20 became misclassified, with none left unresolved. This is the exact near-miss case documented in the paper (`computer vision` versus `computer imaging and vision`) occurring naturally at scale and being fully recovered by the fix.

1.5.2 Qwen3-8B, abstracts, 2500 documents

Table 3 shows the same setup using abstracts instead of titles as the document representation.

Table 3: Qwen3-8B (Ollama backend), abstracts, 2500 documents

	Success	Misclassified	Hallucination
Old matching	61.8%	29.5%	8.6%
New matching	66.3%	30.5%	3.2%

The same `computer vision` pattern appeared in the abstracts run: 136 of 2500 answers (5.4%) were the literal string `computer vision`, all hallucinations under old matching, recovered to 111 successes and 25 misclassified under new matching.

Unlike the original paper, abstracts did not produce more hallucinations than titles for this model (8.6% versus 7.2% under old matching, 3.2% versus 3.1% under new matching, both essentially flat). The paper reported a jump from 25% to 33% hallucination when moving from titles to abstracts. Success and misclassification rates are also roughly flat between titles and abstracts for this model, so Qwen3-8B does not reproduce the paper's title-versus-abstract effect at all.

1.5.3 Claude

A further full 2500-document run was completed for Claude, covering both titles and abstracts. Table 4 reports the results under new matching. The abstract-field script attempted prompt caching, but every prompt fell under Anthropic's minimum cacheable size, so caching never actually activated. This affected only cost and latency, not the answers themselves.

Table 4: Claude, 2500 documents, new matching

Run	Success	Misclassified	Hallucination
Claude, titles	70.9%	27.6%	1.5%
Claude, abstracts	74.0%	23.3%	2.7%

- **Hallucination collapses for Claude, regardless of document representation.** The paper reported 25% (titles) and 33% (abstracts). Claude stays under 3% on both. Combined with Qwen3-8B's own low hallucination rate (Table 19), this looks like a model-generation effect rather than a title-versus-abstract effect: newer, better instruction-following models simply produce far less unparseable output in the first place.
- **Claude favors abstracts over titles.** This later turned out to match Qwen3-8B's own result once run at native bf16 precision (Section 1.6), rather than being an outlier.

For infrastructure completeness: Qwen3-8B was also run through GGUF Q4_K_M via llama.cpp before settling on native Hugging Face `transformers` as the backend used throughout this section. It is documented in Table 19 for completeness, not treated as a separate finding here. (The Ollama backend is discussed separately in Section 1.4, it produced the dataset used for the matching-fix demonstration above, not a minor side attempt.)

1.5.4 Overall results comparison

Table 5 summarizes every run of the newer-model quality check alongside the paper's original baseline.

Table 5: Newer-model quality check: overall results comparison

Run	Success	Misclassified	Hallucination
Paper (Llama 3 8B, titles, 2500 docs, 5 runs)	50%	25%	25%
Paper (Llama 3 8B, abstracts, 2500 docs, 5 runs)	50%	17%	33%
Qwen3-8B local, 3-turn, small sample	40%	60%	0%
Qwen3-8B local, single-prompt, small sample	55%	40%	5%
Qwen3-8B (Ollama), single-prompt, 2500 docs, titles	66.6%	30.3%	3.1%
Qwen3-8B (Ollama), single-prompt, 2500 docs, abstracts	66.3%	30.5%	3.2%
Qwen3-8B (llama.cpp), 2500 docs, titles	67.5%	29.8%	2.8%
Qwen3-8B (llama.cpp), 2500 docs, abstracts	66.5%	30.3%	3.2%
Claude, 2500 docs, titles	70.9%	27.6%	1.5%
Claude, 2500 docs, abstracts	74.0%	23.3%	2.7%

At this point the newer-model quality check was complete: every newer model tested beat the paper's Llama 3 8B baseline by a wide margin, driven mostly by a collapse in hallucination rate. This raised a direct follow-up question. Qwen3-8B (8 billion parameters) and Claude (size undisclosed, presumably much larger) both beat a same-generation-class Llama 3 8B badly, so is this improvement really about newer training, or could a big part of it just be model size? The rest of this section answers that question directly, by holding the model family fixed and varying only size.

1.6 Extending to Model Size: the Qwen3.5 Family

1.6.1 Setup

To isolate size as a variable, the newer Qwen3.5 model family was run at five checkpoint sizes (0.8B, 2B, 4B, 9B, 27B), using the identical task, prompt, matching logic, and 2500-document dataset as above, so results stay directly comparable.

- Runs were performed on a Helmholtz cluster node: an NVIDIA RTX 4090 (24GB VRAM), an AMD Ryzen 9 7900X (12 cores, 24 threads), and 125GB of system RAM.
- A Hugging Face `transformers` backend (`AutoModelForCausalLM`) was used, with model size and input field passed as command-line arguments rather than hardcoded, so each size runs as an independent process and a crash or out-of-memory error on the largest checkpoint cannot take the others down with it.
- A reusable metrics module was built to record model load time, total runtime, average and median per-call latency, tokens per second, truncation rate, and peak RAM and GPU memory, saved separately from the accuracy results and cross-referenced to them explicitly.
- A real bug was found and fixed during setup: `tokenizer.apply_chat_template(..., return_tensors="pt")` returned a `BatchEncoding` object rather than a bare tensor on the installed `transformers` version. Passing it directly into `model.generate()` raised an `AttributeError` deep inside the generation internals. This was fixed by requesting `return_dict=True` explicitly and unpacking the dictionary into `generate()`.

A known constraint going in: rough bf16 memory arithmetic (2 bytes per parameter) puts the 27B checkpoint at roughly 54GB, more than double the 24GB available on this single GPU. Since only one GPU is available, `device_map="auto"` cannot spread the model across multiple devices, so it either fails outright or silently offloads the overflow to system RAM, which is dramatically slower and not a fair comparison point. This constraint, and how it was ultimately resolved, is discussed below.

Each size and field combination was additionally run more than once as a consistency check, to rule out run-to-run fluctuation before treating any single number as representative. No fluctuation was found at all: every repeat at fixed settings came back bit-identical to the first run, not just close, exactly the same success, misclassified, and hallucination counts every time. This is expected once traced to the cause: generation used greedy decoding (`do_sample=False`) throughout, which is fully deterministic, so there is no stochastic variance in this pipeline for repeat runs to reveal. The repeat runs instead ended up useful for a different, unplanned reason, detailed in Section 1.9: several of them were run under 4-bit quantization rather than the original precision, which turned the intended consistency check into a genuine precision comparison instead.

1.7 Qwen3.5 Size Sweep Results

1.7.1 Qwen3.5-4B

Table 6: Qwen3.5-4B, 2500 documents, new matching

	Success	Misclassified	Hallucination
Titles	75.4%	24.0%	0.6%
Abstracts	76.7%	22.3%	1.0%

Qwen3.5-4B, at less than half the parameter count of Qwen3-8B, beats both Qwen3-8B (68.2% / 70.0%, native bf16 precision) and Claude (70.9% / 74.0%) on success rate. This is the first direct evidence that newer generation matters more than raw size, at least at this data point. It also matches the title-versus-abstract direction of both Qwen3-8B and Claude at native precision (abstracts slightly ahead in every case).

1.7.2 Qwen3.5-0.8B

Table 7: Qwen3.5-0.8B, 2500 documents, new matching

	Success	Misclassified	Hallucination
Titles	52.1%	47.3%	0.6%
Abstracts	53.7%	43.9%	2.4%

This is a real drop from 4B, roughly 23 points lower on success, so size clearly does matter within the family. The interesting nuance is where the drop shows up: hallucination stays low even at 0.8B (0.6% / 2.4%, close to 4B's own rate), while misclassification jumps sharply (47.3% / 43.9%, versus 24.0% / 22.3% at 4B). Staying inside the controlled vocabulary appears to be a low bar that even a very small model clears; picking the correct topic is what actually requires the additional capacity. Even so, 0.8B (52.1% on titles) still narrowly beats the original paper's Llama 3 8B baseline (50%), a model roughly ten times its size from an older generation.

1.7.3 Qwen3.5-9B

Table 8: Qwen3.5-9B, 2500 documents, new matching

	Success	Misclassified	Hallucination
Titles	78.6%	20.8%	0.6%
Abstracts	80.8%	18.7%	0.5%

This was the best result across the whole size sweep. Table 9 shows the scaling picture across the three sizes run to that point.

Table 9: Scaling picture, three Qwen3.5 sizes, new matching success rate

Size	Titles	Abstracts
0.8B	52.1%	53.7%
4B	75.4%	76.7%
9B	78.6%	80.8%

Bigger keeps winning at this point, but with clear diminishing returns: the jump from 0.8B to 4B is roughly 23 points, while 4B to 9B is only 3 to 4 points. Most of the achievable gain from scale was already captured by 4B.

1.7.4 Qwen3.5-27B: the VRAM constraint in practice

As anticipated, 27B did not fit in bf16 on the 24GB GPU. `device_map="auto"` split the model across GPU and system RAM automatically rather than failing outright. Two title runs were completed under this configuration before switching to 4-bit quantization.

Table 10: Qwen3.5-27B, titles, bf16, CPU-offloaded (two runs)

	Success	Misclassified	Hallucination
Run 1	75.4%	24.2%	0.4%
Run 2	75.4%	24.2%	0.4%

Both runs produced identical counts, expected since generation uses greedy decoding (`do_sample=False`) and is fully deterministic; this confirms pipeline reproducibility rather than providing two independent accuracy samples.

The runtime cost of CPU offloading was severe rather than marginal: approximately 25,263 and 24,792 seconds per run (roughly 7 hours each), at 0.38 and 0.37 tokens per second, compared to 9B's approximately 28 tokens per second, a roughly 73-fold throughput collapse and 30-fold wall-clock slowdown. Peak GPU memory reached approximately 21GB, meaning most of the model did fit on the GPU, while peak system RAM reached approximately 53GB for the remainder, shuffled in layer by layer on every forward pass.

The accuracy result was genuinely unexpected: 27B (75.4%) did not beat 9B (78.6%), and landed exactly on 4B's number (75.4%). The clean monotonic scaling seen through 0.8B, 4B, and 9B breaks at this point. Because both runs agreed exactly, this was treated as a real result rather than noise, and the decision was made to switch to `bitsandbytes` 4-bit quantization for further 27B runs, since the full model then fits on the GPU (approximately 13.5GB) and avoids the CPU-offload penalty entirely. The two bf16 runs above were kept as a separate quantization comparison point rather than folded into the main run set below.

All three abstract runs landed on exactly the same counts, again confirming determinism, with timing tightly consistent across the three (1316.5s, 1258.8s, 1262.8s; roughly 21 to 22 minutes each; approximately 8.0 tokens per second; approximately 18.9GB peak GPU memory each time).

Table 11: Qwen3.5-27B, 4-bit, GPU-only (title x1, abstract x3, new matching)

	Success	Misclassified	Hallucination
Titles	76.0%	24.0%	0.04%
Abstracts (3 runs, identical)	77.0%	22.9%	0.1%

On speed, the title run completed in 1003 seconds (about 16.7 minutes, 9.6 tokens per second), a roughly 25-fold speedup over the bf16 CPU-offloaded version, but still about 1.5 times slower than 9B (about 11 minutes, 28 tokens per second). This is despite 4-bit quantization needing less raw memory bandwidth per token than 9B's bf16, since single-sequence decoding is memory-bandwidth-bound rather than purely compute-bound: a 27B model at roughly 0.5 bytes per parameter moves less data per token than a 9B model at 2 bytes per parameter. The gap that remains comes from dequantization compute overhead, the cost of unpacking 4-bit weights before each matrix multiplication, rather than from bandwidth.

On accuracy, the aggregate success rate barely moved between precisions (75.4% bf16 to 76.0% 4-bit on titles), but a direct per-document comparison between the two title runs showed only 87.8% identical raw answers; 12.2% (304 of 2500 documents) received a genuinely different answer under quantization. Of those 304, 99 stayed success with a different valid label, 73 moved from misclassified to success, 62 stayed misclassified with a different wrong label, 59 moved from success to misclassified, 7 moved from hallucination to misclassified, 3 moved from hallucination to success, and 1 moved from success to hallucination. The net effect (83 improved against 60 worsened) is consistent with the small aggregate gain once wash cases are netted out. This is a useful methodological caution: an aggregate accuracy number that looks stable can still hide substantial churn at the level of individual answers.

With both bf16 and 4-bit precision now tested, the central finding held up under replication rather than being a single-run artifact: 27B (76.0% / 77.0%) still did not beat 9B (78.6% / 80.8%) at either field or either precision. Table 12 shows the complete scaling picture.

Table 12: Full Qwen3.5 scaling picture, new matching success rate

Size	Titles	Abstracts
0.8B	52.1%	53.7%
4B	75.4%	76.7%
9B	78.6%	80.8%
27B	76.0%	77.0%

Within this model family, on this task, **9B achieves the highest accuracy**, not the largest checkpoint tested. Scaling helps sharply from 0.8B to 4B, keeps helping a little through 9B, and then reverses at 27B. This framing is preferred in the thesis over a flat bigger-is-better narrative, since the data directly contradicts that story at the top end.

1.8 Overall Comparison Across Both Investigations

Table 13 brings together every run from the newer-model quality check and the size sweep alongside the paper's original baseline.

Table 13: Overall comparison, all models and sizes, new matching

Run	Success	Misclassified	Hallucination
Paper (Llama 3 8B), titles	50%	25%	25%
Paper (Llama 3 8B), abstracts	50%	17%	33%
Qwen3.5-0.8B, titles	52.1%	47.3%	0.6%
Qwen3.5-0.8B, abstracts	53.7%	43.9%	2.4%
Qwen3-8B, titles	68.2%	29.3%	2.5%
Qwen3-8B, abstracts	70.0%	27.2%	2.8%
Claude, titles	70.9%	27.6%	1.5%
Claude, abstracts	74.0%	23.3%	2.7%
Qwen3.5-4B, titles	75.4%	24.0%	0.6%
Qwen3.5-4B, abstracts	76.7%	22.3%	1.0%
Qwen3.5-9B, titles	78.6%	20.8%	0.6%
Qwen3.5-9B, abstracts	80.8%	18.7%	0.5%
Qwen3.5-27B, titles, bf16/CPU-offload (x2, identical)	75.4%	24.2%	0.4%
Qwen3.5-27B, titles, 4-bit	76.0%	24.0%	0.04%
Qwen3.5-27B, abstracts, 4-bit (x3, identical)	77.0%	22.9%	0.1%

1.9 Quantization Sensitivity Scales Inversely With Model Size

A script configuration issue during the consistency-checking pass applied 4-bit quantization to every model size, not only 27B. Because generation is fully deterministic under greedy decoding, and every repeat at fixed settings had already come back bit-identical throughout this work, the originally intended repeat-run variance check was not actually testable, there is no run-to-run randomness in this pipeline to measure. What the resulting runs provided instead was more informative: one native-precision run plus two confirmed-stable 4-bit runs per model, giving a genuine bf16-versus-4-bit comparison across the full size range. Table 14 summarizes it.

Table 14: Quantization sensitivity by model size, new matching success rate

Size	Field	bf16	4-bit	Difference
0.8B	titles	52.1%	39.2%	**-12.9 pts**
0.8B	abstracts	53.7%	27.6%	**-26.1 pts**
4B	titles	75.4%	74.6%	-0.8 pts
4B	abstracts	76.7%	74.5%	-2.2 pts
9B	titles	78.6%	80.0%	+1.4 pts
9B	abstracts	80.8%	81.5%	+0.7 pts
27B	titles	75.4% (bf16, CPU-offloaded)	76.0%	+0.6 pts
27B	abstracts	n/a (4-bit only)	77.0%	n/a

The pattern is clean and monotonic: quantization sensitivity drops sharply as model size increases. The

0.8B checkpoint is fragile, losing 13 to 26 points, more than half of its edge over random guessing on abstracts. By 4B the effect is mostly absorbed, under 2.5 points lost. At 9B and 27B the model is effectively indifferent to 4-bit quantization, with gains and losses both staying under 1.5 points, likely noise level. This reads as larger models having more redundant capacity to tolerate precision loss, while smaller ones have no slack to spare.

A secondary supporting data point comes from Qwen3-8B (the older, non-3.5 generation), which was run under three different precisions on the identical checkpoint: native bf16 (68.2% / 70.0%, hallucination 2.5% / 2.8%), GGUF Q4_K_M via llama.cpp (67.5% / 66.5%, hallucination 2.8% / 3.2%), and `bitsandbytes` 4-bit (64.0% / 63.1%, hallucination 7.8% / 7.0%). Native precision is best, GGUF's 4-bit is close behind, and `bitsandbytes`'s 4-bit is meaningfully worse on both success and hallucination. Same checkpoint, same task, three outcomes purely from precision and quantization algorithm choice. This is consistent with older or less robustly trained checkpoints being generally less tolerant of quantization noise than the Qwen3.5 sizes above, not only a pure model-size effect, and it reinforces the same lesson from a different angle: quantization method, not just bit-width, determines how much accuracy is actually lost.

1.10 Per-Model Raw Run Tables

Every individual run is listed below, under new matching. Every duplicate run at the same field and precision was directly re-verified as bit-identical, not assumed, consistent with the deterministic decoding used throughout.

Table 15: Qwen3.5-0.8B, all runs

Field	Precision	Success	Misclassified	Hallucination
title	bf16	52.1%	47.3%	0.6%
title	4-bit	39.2%	58.8%	2.0%
title	4-bit	39.2%	58.8%	2.0%
abstract	bf16	53.7%	43.9%	2.4%
abstract	4-bit	27.6%	69.6%	2.8%
abstract	4-bit	27.6%	69.6%	2.8%

Table 16: Qwen3.5-4B, all runs

Field	Precision	Success	Misclassified	Hallucination
title	bf16	75.4%	24.0%	0.6%
title	4-bit	74.6%	24.8%	0.5%
title	4-bit	74.6%	24.8%	0.5%
abstract	bf16	76.7%	22.3%	1.0%
abstract	4-bit	74.5%	24.3%	1.2%
abstract	4-bit	74.5%	24.3%	1.2%

Table 17: Qwen3.5-9B, all runs

Field	Precision	Success	Misclassified	Hallucination
title	bf16	78.6%	20.8%	0.6%
title	4-bit	80.0%	19.7%	0.2%
title	4-bit	80.0%	19.7%	0.2%
abstract	bf16	80.8%	18.7%	0.5%
abstract	4-bit	81.5%	17.6%	0.9%
abstract	4-bit	81.5%	17.6%	0.9%

Table 18: Qwen3.5-27B, all runs

Field	Precision	Success	Misclassified	Hallucination
title	bf16 (CPU-offload, 7hr)	75.4%	24.2%	0.4%
title	bf16 (CPU-offload, 7hr)	75.4%	24.2%	0.4%
title	4-bit	76.0%	24.0%	0.04%
abstract	4-bit	77.0%	22.9%	0.1%
abstract	4-bit	77.0%	22.9%	0.1%
abstract	4-bit	77.0%	22.9%	0.1%

1.11 Status

Both investigations are complete. The newer-model quality check found that every newer model tested beat the paper's Llama 3 8B baseline by a wide margin, driven mostly by a collapse in hallucination rate rather than a shift in raw success rate. The size sweep found that this improvement is not simply a function of parameter count: scaling within the Qwen3.5 family helps sharply up to 4B, keeps helping modestly to 9B, and then reverses at 27B, with 9B achieving the highest accuracy of the sizes tested. A further, unplanned finding showed that quantization sensitivity itself scales inversely with model size, with the smallest checkpoint losing over a quarter of its accuracy edge under 4-bit quantization while the largest checkpoints were unaffected or slightly favorable. Remaining future work: Qwen3.5-2B has not yet been run, and would fill in the one gap in the size sweep between 0.8B and 4B.

Table 19: Qwen3-8B (non-3.5), all runs

Field	Precision	Success	Misclassified	Hallucination
title	bf16 (native)	68.2%	29.3%	2.5%
title	GGUF Q4_K_M	67.5%	29.8%	2.8%
title	4-bit (bnb)	64.0%	28.2%	7.8%
title	4-bit (bnb)	64.0%	28.2%	7.8%
abstract	bf16 (native)	70.0%	27.2%	2.8%
abstract	GGUF Q4_K_M	66.5%	30.3%	3.2%
abstract	4-bit (bnb)	63.1%	29.8%	7.0%
abstract	4-bit (bnb)	63.1%	29.8%	7.0%